



# Reinforcement Learning

---

P Satya Jayadev

Web Enabled M.Tech,  
Indian Institute of Technology Madras

# Topic 4

## Value Function Approximation



1. Why Value Function Approximation?
2. Features, Parameters and Generalisation
3. Learning from Monte Carlo Targets
4. Linear Function Approximation
5. Learning from TD Targets: Semi-Gradient TD
6. Control with Value Function Approximation
7. Non-Linear Function Approximation
8. Stability and the Deadly Triad



## Recall: Value Function Representation

- ▶ RL algorithms discussed until now assume value function in a tabular form i.e., separate value associated with each state or state-action pair



# Recall: Value Function Representation

- ▶ RL algorithms discussed until now assume value function in a tabular form i.e., separate value associated with each state or state-action pair
- ▶ Drawbacks of Tabular Value Function Representation
  - **Scalability:** Not scalable to large and/or continuous state and action spaces
  - **Memory Intensive:** Need to store a value for every state or state-action pair may require lot of memory
  - **Lack of Generalisation:** Values cannot be generalised or extrapolated from seen to unseen states or state-action pairs
  - **Slow Learning:** Lack of generalisation means learning values of all states and state-action pairs requires more samples and takes more time to converge



# Recall: Value Function Representation

- ▶ RL algorithms discussed until now assume value function in a tabular form i.e., separate value associated with each state or state-action pair
- ▶ **Drawbacks of Tabular Value Function Representation**
  - **Scalability:** Not scalable to large and/or continuous state and action spaces
  - **Memory Intensive:** Need to store a value for every state or state-action pair may require lot of memory
  - **Lack of Generalisation:** Values cannot be generalised or extrapolated from seen to unseen states or state-action pairs
  - **Slow Learning:** Lack of generalisation means learning values of all states and state-action pairs requires more samples and takes more time to converge
- ▶ All drawbacks can be addressed when value functions are approximated using a parametric function
- ▶ Types of function approximators include linear functions, polynomial functions, neural networks, decision trees, etc.



# Why Shared Parameters Matter

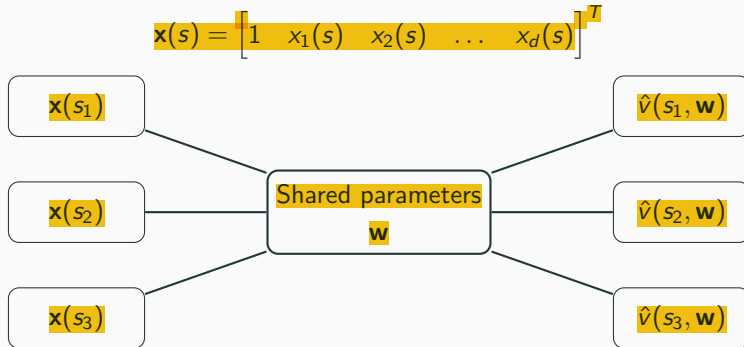
- ▶ Suppose nearby states have similar characteristics.
- ▶ Instead of learning each state independently, we describe every state using features.

$$\mathbf{x}(s) = \begin{bmatrix} 1 & x_1(s) & x_2(s) & \dots & x_d(s) \end{bmatrix}^T$$



# Why Shared Parameters Matter

- Suppose nearby states have similar characteristics.
- Instead of learning each state independently, we describe every state using features.



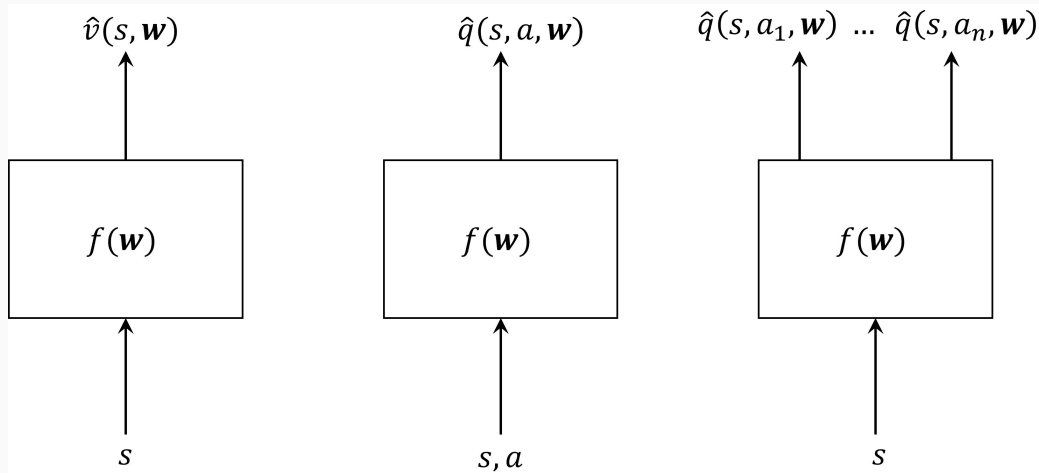
## Generalisation

An update based on experience at one state changes  $\mathbf{w}$ . Because the same  $\mathbf{w}$  is used for all states, estimates for other similar states may also change.





## Types of Value Function Approximators



# Regression Formulation of Value Function Approximators

## ► Type 1:

- **Inputs:** State Variables for a particular state
- **Outputs:** State Values for a particular state (Single output)
- **Data:** State variables mapped to target value of the state



# Regression Formulation of Value Function Approximators

## ► Type 1:

- **Inputs:** State Variables for a particular state
- **Outputs:** State Values for a particular state (Single output)
- **Data:** State variables mapped to target value of the state

## ► Type 2:

- **Inputs:** State and Action Variables of a state-action pair
- **Outputs:** Action or Q-Values of a state-action pair (Single output)
- **Data:** State and action variables mapped to target Q-value of state-action pair



# Regression Formulation of Value Function Approximators

## ► Type 1:

- **Inputs:** State Variables for a particular state
- **Outputs:** State Values for a particular state (Single output)
- **Data:** State variables mapped to target value of the state

## ► Type 2:

- **Inputs:** State and Action Variables of a state-action pair
- **Outputs:** Action or Q-Values of a state-action pair (Single output)
- **Data:** State and action variables mapped to target Q-value of state-action pair

## ► Type 3:

- **Inputs:** State Variables for a particular state
- **Outputs:** Action or Q-Values for every state-action pair (Multiple outputs)
- **Data:** State and action variables mapped to target Q-values of each state-action pair



# Objective Function for Value Approximation

- Ideally, we want the approximated value to be close to the true value under policy  $\pi$ :

$$J(\mathbf{w}) = \mathbb{E}_{s \sim \mu} \left[ (v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

For a finite state space:

$$J(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$



# Objective Function for Value Approximation

- Ideally, we want the approximated value to be close to the true value under policy  $\pi$ :

$$J(\mathbf{w}) = \mathbb{E}_{s \sim \mu} \left[ (v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

For a finite state space:

$$J(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

- $\mu(s)$  specifies how much importance is given to accurate prediction at state  $s$ .
- Frequently visited states may receive greater weight than rarely visited states.
- The objective is to find parameters  $\mathbf{w}$  that minimise  $J(\mathbf{w})$ .



# Objective Function for Value Approximation

- Ideally, we want the approximated value to be close to the true value under policy  $\pi$ :

$$J(\mathbf{w}) = \mathbb{E}_{s \sim \mu} \left[ (v_{\pi}(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

For a finite state space:

$$J(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) [v_{\pi}(s) - \hat{v}(s, \mathbf{w})]^2$$

- $\mu(s)$  specifies how much importance is given to accurate prediction at state  $s$ .
- Frequently visited states may receive greater weight than rarely visited states.
- The objective is to find parameters  $\mathbf{w}$  that minimise  $J(\mathbf{w})$ .

## Practical Difficulty

The true value  $v_{\pi}(s)$  is generally unknown and cannot be directly used as a training target.



# Practical Learning Targets

- ▶ The true value  $v_\pi(S_t)$  is generally unknown.
- ▶ Therefore, we replace it with a sample-based target  $U_t$  obtained from experience.





# Practical Learning Targets

- ▶ The true value  $v_\pi(S_t)$  is generally unknown.
- ▶ Therefore, we replace it with a sample-based target  $U_t$  obtained from experience.

| Method      | Target $U_t$                                            | When available       |
|-------------|---------------------------------------------------------|----------------------|
| Monte Carlo | $U_t = G_t$                                             | End of the episode   |
| TD(0)       | $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$ | After one transition |
| n-step TD   | $U_t = G_t^{(n)}$                                       | After n transitions  |



# Practical Learning Targets

- ▶ The true value  $v_{\pi}(S_t)$  is generally unknown.
- ▶ Therefore, we replace it with a sample-based target  $U_t$  obtained from experience.

| Method      | Target $U_t$                                            | When available       |
|-------------|---------------------------------------------------------|----------------------|
| Monte Carlo | $U_t = G_t$                                             | End of the episode   |
| TD(0)       | $U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$ | After one transition |
| n-step TD   | $U_t = G_t^{(n)}$                                       | After n transitions  |

## Common Learning Principle

Adjust  $\mathbf{w}$  so that the current estimate  $\hat{v}(S_t, \mathbf{w})$  moves closer to the available target  $U_t$ .



# Learning the Parameters Using Stochastic Gradient Descent

- ▶ At time  $t$ , the approximator produces the current estimate:

$$\hat{v}(S_t, \mathbf{w}_t)$$

- ▶ A sample-based target  $U_t$  is obtained from MC, TD, or an  $n$ -step method.



# Learning the Parameters Using Stochastic Gradient Descent

- ▶ At time  $t$ , the approximator produces the current estimate:

$$\hat{v}(S_t, \mathbf{w}_t)$$

- ▶ A sample-based target  $U_t$  is obtained from MC, TD, or an  $n$ -step method.
- ▶ Define the sample loss:

$$L_t(\mathbf{w}) = \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2$$



# Learning the Parameters Using Stochastic Gradient Descent

- ▶ At time  $t$ , the approximator produces the current estimate:

$$\hat{v}(S_t, \mathbf{w}_t)$$

- ▶ A sample-based target  $U_t$  is obtained from MC, TD, or an  $n$ -step method.
- ▶ Define the sample loss:

$$L_t(\mathbf{w}) = \frac{1}{2} [U_t - \hat{v}(S_t, \mathbf{w})]^2$$

Gradient descent update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla_{\mathbf{w}} L_t(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

## Interpretation

Weight change = learning rate  $\times$  prediction error  $\times$  sensitivity of the prediction



# Algorithm: Gradient Monte-Carlo for Estimating $V_\pi$

## Input:

- ▶ Policy  $\pi$  to be evaluated
- ▶ Differentiable value approximator  $\hat{v}(s, \mathbf{w})$
- ▶ Learning rate  $\alpha$

**Initialise:**  $\mathbf{w} \in \mathbb{R}^d$



# Algorithm: Gradient Monte-Carlo for Estimating $V_\pi$

## Input:

- ▶ Policy  $\pi$  to be evaluated
- ▶ Differentiable value approximator  $\hat{v}(s, \mathbf{w})$
- ▶ Learning rate  $\alpha$

**Initialise:**  $\mathbf{w} \in \mathbb{R}^d$  **For each episode generated by following  $\pi$ :**

1. Observe the trajectory:  $S_0, A_0, R_1, S_1, \dots, S_T$



# Algorithm: Gradient Monte-Carlo for Estimating $V_\pi$

## Input:

- ▶ Policy  $\pi$  to be evaluated
- ▶ Differentiable value approximator  $\hat{v}(s, \mathbf{w})$
- ▶ Learning rate  $\alpha$

**Initialise:**  $\mathbf{w} \in \mathbb{R}^d$  **For each episode generated by following  $\pi$ :**

1. Observe the trajectory:  $S_0, A_0, R_1, S_1, \dots, S_T$
2. For each time step  $t = 0, 1, \dots, T - 1$ , compute the return:  $G_t = R_{t+1} + \gamma R_{t+2} + \dots$





# Algorithm: Gradient Monte-Carlo for Estimating $V_\pi$

## Input:

- ▶ Policy  $\pi$  to be evaluated
- ▶ Differentiable value approximator  $\hat{v}(s, \mathbf{w})$
- ▶ Learning rate  $\alpha$

**Initialise:**  $\mathbf{w} \in \mathbb{R}^d$  **For each episode generated by following  $\pi$ :**

1. Observe the trajectory:  $S_0, A_0, R_1, S_1, \dots, S_T$
2. For each time step  $t = 0, 1, \dots, T - 1$ , compute the return:  $G_t = R_{t+1} + \gamma R_{t+2} + \dots$
3. Update the shared parameters:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$



## Algorithm: Gradient Monte-Carlo for Estimating $V_\pi$

### Input:

- ▶ Policy  $\pi$  to be evaluated
- ▶ Differentiable value approximator  $\hat{v}(s, \mathbf{w})$
- ▶ Learning rate  $\alpha$

**Initialise:**  $\mathbf{w} \in \mathbb{R}^d$  **For each episode generated by following  $\pi$ :**

1. Observe the trajectory:  $S_0, A_0, R_1, S_1, \dots, S_T$
2. For each time step  $t = 0, 1, \dots, T - 1$ , compute the return:  $G_t = R_{t+1} + \gamma R_{t+2} + \dots$
3. Update the shared parameters:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

### Key Property

The MC target  $G_t$  does not depend on the current parameters  $\mathbf{w}$ , so this is a true stochastic-gradient update.



# Linear Function Approximation

- Represent each state using a feature vector:

$$\mathbf{x}(s) = \begin{bmatrix} 1 & x_1(s) & x_2(s) & \dots & x_d(s) \end{bmatrix}^T$$

- Approximate the value using a function that is linear in the weights:

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$$



# Linear Function Approximation

- Represent each state using a feature vector:

$$\mathbf{x}(s) = \begin{bmatrix} 1 & x_1(s) & x_2(s) & \dots & x_d(s) \end{bmatrix}^T$$

- Approximate the value using a function that is linear in the weights:

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$$

- Gradient of value function with respect to the weights is:

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$



# Linear Function Approximation

- Represent each state using a feature vector:

$$\mathbf{x}(s) = \begin{bmatrix} 1 & x_1(s) & x_2(s) & \dots & x_d(s) \end{bmatrix}^T$$

- Approximate the value using a function that is linear in the weights:

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$$

- Gradient of value function with respect to the weights is:

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$

## Important Clarification

The approximator must be linear in  $\mathbf{w}$ , but the features themselves may be non-linear functions of the state.



# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- Feature vector:  $\mathbf{x}(s_3) = ?$



# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- ▶ Feature vector:  $\mathbf{x}(s_3) = ?$
- ▶ Initial weights:  $\mathbf{w}_0 = [0.2, 0.5]$



# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- ▶ Feature vector:  $\mathbf{x}(s_3) = ?$
- ▶ Initial weights:  $\mathbf{w}_0 = [0.2, 0.5]$
- ▶ Initial value:  $\hat{v}(s_3, \mathbf{w}) = ?$





# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- ▶ Feature vector:  $\mathbf{x}(s_3) = ?$
- ▶ Initial weights:  $\mathbf{w}_0 = [0.2, 0.5]$
- ▶ Initial value:  $\hat{v}(s_3, \mathbf{w}) = ?$
- ▶ MC Target:  $G(s_3) = 0.8$
- ▶ Learning rate:  $\alpha = 0.1$

**Update Rule:**  $\mathbf{w} \leftarrow \mathbf{w} + \alpha(G(s_3) - \hat{v}(s_3, \mathbf{w}))\mathbf{x}(s_3)$



# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- ▶ Feature vector:  $\mathbf{x}(s_3) = ?$
- ▶ Initial weights:  $\mathbf{w}_0 = [0.2, 0.5]$
- ▶ Initial value:  $\hat{v}(s_3, \mathbf{w}) = ?$
- ▶ MC Target:  $G(s_3) = 0.8$
- ▶ Learning rate:  $\alpha = 0.1$

**Update Rule:**  $\mathbf{w} \leftarrow \mathbf{w} + \alpha(G(s_3) - \hat{v}(s_3, \mathbf{w}))\mathbf{x}(s_3)$

**Updated Weights:**  $\mathbf{w}_1 = ?$



# Linear Function Approximation: Example

**Environment:** 1D Gridworld with 5 states:  $s_1$  to  $s_5$

Each state is represented using a feature vector:  $\mathbf{x}(s) = [1, \text{state index}/5]$

**Objective:** Estimate  $v(s_3)$  using a linear approximator after one MC target based update

**Given:**

- ▶ Feature vector:  $\mathbf{x}(s_3) = ?$
- ▶ Initial weights:  $\mathbf{w}_0 = [0.2, 0.5]$
- ▶ Initial value:  $\hat{v}(s_3, \mathbf{w}) = ?$
- ▶ MC Target:  $G(s_3) = 0.8$
- ▶ Learning rate:  $\alpha = 0.1$

**Update Rule:**  $\mathbf{w} \leftarrow \mathbf{w} + \alpha(G(s_3) - \hat{v}(s_3, \mathbf{w}))\mathbf{x}(s_3)$

**Updated Weights:**  $\mathbf{w}_1 = ? [0.23, 0.518]$



# Effect of the Update on Other States

The update was based only on experience from  $s_3$ :

$$\mathbf{w}_0 = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix} \longrightarrow \mathbf{w}_1 = \begin{bmatrix} 0.23 \\ 0.518 \end{bmatrix}$$

Since every state uses the same weights,

$$\hat{v}(s_i, \mathbf{w}) = w_1 + w_2 \left( \frac{i}{5} \right),$$

the estimated values of all states change.



# Effect of the Update on Other States

The update was based only on experience from  $s_3$ :

$$\mathbf{w}_0 = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix} \longrightarrow \mathbf{w}_1 = \begin{bmatrix} 0.23 \\ 0.518 \end{bmatrix}$$

Since every state uses the same weights,

$$\hat{v}(s_i, \mathbf{w}) = w_1 + w_2 \left( \frac{i}{5} \right),$$

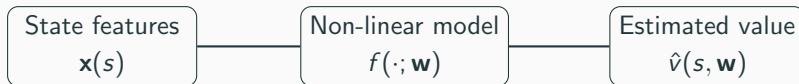
the estimated values of all states change.

| State | Before update | After update |
|-------|---------------|--------------|
| $s_1$ | 0.3000        | 0.3336       |
| $s_2$ | 0.4000        | 0.4372       |
| $s_3$ | 0.5000        | 0.5408       |
| $s_4$ | 0.6000        | 0.6444       |
| $s_5$ | 0.7000        | 0.7480       |



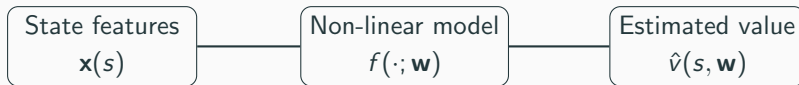
# Non-Linear Value Function Approximation

- ▶ A linear approximator may be insufficient when the relationship between state features and value is complex.
- ▶ A non-linear model can learn interactions and complex patterns directly from the state representation:



# Non-Linear Value Function Approximation

- ▶ A linear approximator may be insufficient when the relationship between state features and value is complex.
- ▶ A non-linear model can learn interactions and complex patterns directly from the state representation:

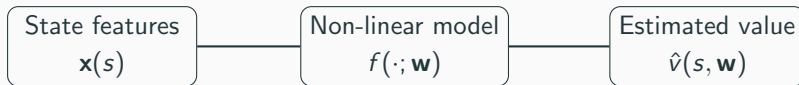


- ▶ Neural networks are the most common choice in modern RL.
- ▶ Other possibilities include kernel methods, radial basis functions, and tree-based models.
- ▶ In a neural network,  $\mathbf{w}$  includes all weights and biases.



# Non-Linear Value Function Approximation

- ▶ A linear approximator may be insufficient when the relationship between state features and value is complex.
- ▶ A non-linear model can learn interactions and complex patterns directly from the state representation:



- ▶ Neural networks are the most common choice in modern RL.
- ▶ Other possibilities include kernel methods, radial basis functions, and tree-based models.
- ▶ In a neural network,  $\mathbf{w}$  includes all weights and biases.

## Key Difference from Linear Approximation

The model can represent more complex value functions, but the optimisation problem is generally non-convex.





# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:



# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:

1. Pass the state features through the network:  $\mathbf{x}(S_t) \rightarrow \hat{v}(S_t, \mathbf{w}_t)$



# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:

1. Pass the state features through the network:  $\mathbf{x}(S_t) \rightarrow \hat{v}(S_t, \mathbf{w}_t)$
2. Compare the prediction with the learning target  $U_t$  and compute error:  $U_t - \hat{v}(S_t, \mathbf{w}_t)$



# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:

1. Pass the state features through the network:  $\mathbf{x}(S_t) \rightarrow \hat{v}(S_t, \mathbf{w}_t)$
2. Compare the prediction with the learning target  $U_t$  and compute error:  $U_t - \hat{v}(S_t, \mathbf{w}_t)$
3. Use backpropagation to compute the gradients:

$$\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$



# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:

1. Pass the state features through the network:  $\mathbf{x}(S_t) \rightarrow \hat{v}(S_t, \mathbf{w}_t)$
2. Compare the prediction with the learning target  $U_t$  and compute error:  $U_t - \hat{v}(S_t, \mathbf{w}_t)$
3. Use backpropagation to compute the gradients:

$$\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

4. Update all network weights and biases.



# Learning a Non-Linear Approximator

The same general update principle is used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For a neural-network approximator, following are the steps followed:

1. Pass the state features through the network:  $\mathbf{x}(S_t) \rightarrow \hat{v}(S_t, \mathbf{w}_t)$
2. Compare the prediction with the learning target  $U_t$  and compute error:  $U_t - \hat{v}(S_t, \mathbf{w}_t)$
3. Use backpropagation to compute the gradients:

$$\nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

4. Update all network weights and biases.

## Limited Guarantees

General convergence guarantees are limited for nonlinear value approximation, particularly when it is combined with bootstrapping and off-policy learning.



# From Ordinary Gradient Updates to Semi-Gradients

For a general sample target  $U_t$ , we used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For Monte Carlo learning:  $U_t = G_t$

- ▶  $G_t$  is obtained from observed rewards.
- ▶ It does not depend on the current weights  $\mathbf{w}_t$ .
- ▶ Therefore, the update is an ordinary stochastic-gradient update.



# From Ordinary Gradient Updates to Semi-Gradients

For a general sample target  $U_t$ , we used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For Monte Carlo learning:  $U_t = G_t$

- ▶  $G_t$  is obtained from observed rewards.
- ▶ It does not depend on the current weights  $\mathbf{w}_t$ .
- ▶ Therefore, the update is an ordinary stochastic-gradient update.

For TD(0):

$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$$





# From Ordinary Gradient Updates to Semi-Gradients

For a general sample target  $U_t$ , we used:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

For Monte Carlo learning:  $U_t = G_t$

- ▶  $G_t$  is obtained from observed rewards.
- ▶ It does not depend on the current weights  $\mathbf{w}_t$ .
- ▶ Therefore, the update is an ordinary stochastic-gradient update.

For TD(0):

$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$$

## New Difficulty

The TD target itself contains the current approximator and therefore depends on  $\mathbf{w}_t$ .



# The Semi-Gradient Idea

During the current update, we:

- ▶ treat the TD target

$$R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$$

as fixed;

- ▶ differentiate only the current-state estimate

$$\hat{v}(S_t, \mathbf{w}_t).$$



# The Semi-Gradient Idea

During the current update, we:

- ▶ treat the TD target

$$R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$$

as fixed;

- ▶ differentiate only the current-state estimate

$$\hat{v}(S_t, \mathbf{w}_t).$$

Therefore:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

## Why is it called “Semi-Gradient”?

We use only part of the full gradient: the target's dependence on  $\mathbf{w}_t$  is ignored during the update.



# Control with Value Function Approximation

- For control, approximate the action-value function:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w})$$

- After observing a transition, update the parameters using:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}_t)$$



# Control with Value Function Approximation

- For control, approximate the action-value function:

$$q_{\pi}(s, a) \approx \hat{q}(s, a, \mathbf{w})$$

- After observing a transition, update the parameters using:

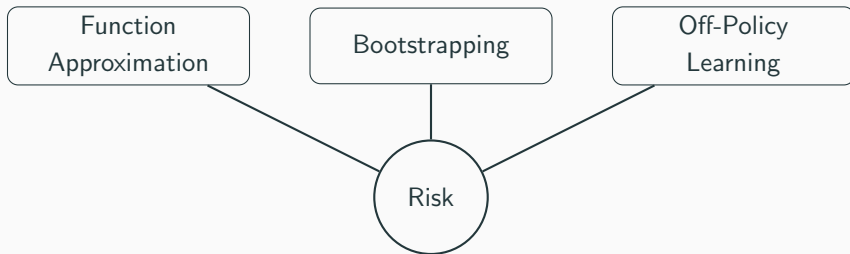
$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}_t)$$

| Method         | Target $U_t$                                                                       |
|----------------|------------------------------------------------------------------------------------|
| SARSA          | $U_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$                   |
| Expected SARSA | $U_t = R_{t+1} + \gamma \sum_a \pi(a   S_{t+1}) \hat{q}(S_{t+1}, a, \mathbf{w}_t)$ |
| Q-learning     | $U_t = R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)$                  |



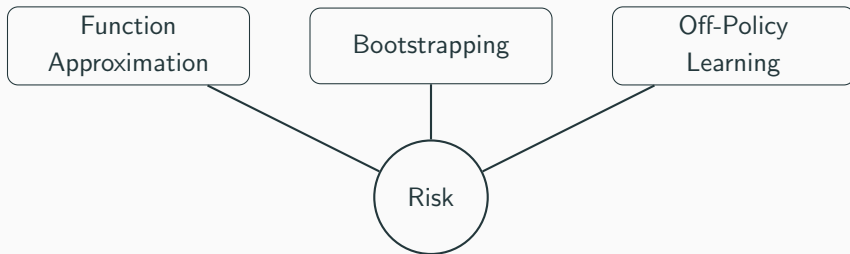
# The Deadly Triad

Instability and divergence are especially likely when the following are combined:



# The Deadly Triad

Instability and divergence are especially likely when the following are combined:



- ▶ Function approximation enables scaling and generalisation.
- ▶ Bootstrapping enables frequent and data-efficient updates.
- ▶ Off-policy learning enables learning from historical data or another behaviour policy.

## Important

None of the three is inherently incorrect. The difficulty arises from their interaction.



# Why Use the Components of the Deadly Triad?

| Component              | Why it is useful                                                                       |
|------------------------|----------------------------------------------------------------------------------------|
| Function approximation | Large and continuous state spaces cannot be handled using tables                       |
| Bootstrapping          | Supports online learning without waiting for the full return                           |
| Off-policy learning    | Allows reuse of logged experience, replay buffers and data collected by other policies |

## Design Challenge

The goal is not simply to remove these components, but to combine them using stable algorithms and careful training practices.





# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.
- ▶ Gradient Monte Carlo uses a target independent of the current weights.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.
- ▶ Gradient Monte Carlo uses a target independent of the current weights.
- ▶ Semi-gradient TD treats the bootstrapped target as fixed during the current update.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.
- ▶ Gradient Monte Carlo uses a target independent of the current weights.
- ▶ Semi-gradient TD treats the bootstrapped target as fixed during the current update.
- ▶ Linear approximation gives a convex objective for gradient MC and has strong convergence properties under suitable assumptions.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.
- ▶ Gradient Monte Carlo uses a target independent of the current weights.
- ▶ Semi-gradient TD treats the bootstrapped target as fixed during the current update.
- ▶ Linear approximation gives a convex objective for gradient MC and has strong convergence properties under suitable assumptions.
- ▶ Non-linear approximators are more expressive but introduce non-convex optimisation and greater stability challenges.



# Summary

- ▶ Function approximation replaces separate table entries with shared parameters:

$$V(s) \longrightarrow \hat{v}(s, \mathbf{w})$$

- ▶ Shared parameters enable generalisation across similar states.
- ▶ Learning uses sample-based targets from MC, TD or  $n$ -step methods.
- ▶ Gradient Monte Carlo uses a target independent of the current weights.
- ▶ Semi-gradient TD treats the bootstrapped target as fixed during the current update.
- ▶ Linear approximation gives a convex objective for gradient MC and has strong convergence properties under suitable assumptions.
- ▶ Non-linear approximators are more expressive but introduce non-convex optimisation and greater stability challenges.
- ▶ Function approximation, bootstrapping and off-policy learning together can create the deadly triad.

